进程的引入

邓一星

2026-03-06 |

顺序执行特点

程序指令按照代码编写的顺序依次执行，前一条指令执行完成后才会开始下一条指令的执行。

顺序执行时，程序独占 CPU、内存等资源，直至执行完毕才会释放资源，系统资源利用率较低。

由于没有并发干扰，程序执行结果具有确定性和可再现性，便于调试和验证。

程序执行过程中只需维护单一的 PC（程序计数器）和寄存器状态，无需复杂的上下文切换机制。

顺序执行特点

特点总结:

- 严格的执行顺序
- 单道程序独占资源
- 可预测的结果
- 简单的程序状态管理

并发执行特点

资源共享与竞争：

并发程序共享 CPU、内存、I/O 设备等资源，可能引发资源竞争问题，需要同步机制保障。

更高的系统吞吐量：

通过重叠 CPU 计算和 I/O 操作，显著提高系统整体资源利用率和任务完成效率。

并发执行特点

宏观并行微观串行：

从用户角度看多个程序同时运行，实际通过时间片轮转等方式在 CPU 上交替执行。

执行结果的不可再现性：

由于执行顺序的不确定性，相同输入的并发程序可能产生不同输出结果。

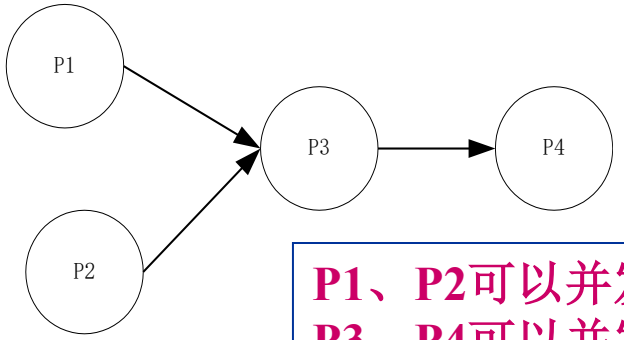
程序并发执行的条件Bernstein

P1：　a=5

P2：　b=6

P3：　c=a+b

P4：　d=c+1



P1、P2可以并发执行吗？
P3、P4可以并发执行呢？

Bernstein条件

读集：R(Pi)={a1,a2,……,am}　程序Pi执行期间参考的变量集合

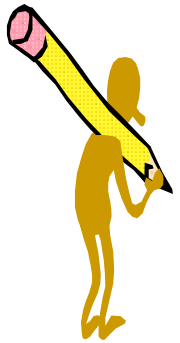
写集：W(Pi)={b1,b2,……,bm}　程序Pi执行期间改变的变量集合

两个进程P1，　P2若满足：

$R(P1)\cap W(P2)\cup R(P2)\cap W(P1)\cup W(P1)\cap W(P2)=\{\}$

则P1，　P2并发执行，且具有可再现性。

Bernstein条件——例1



P1：　a=5

P2:　b=6

R(P1)={}　　W(P1)={a}

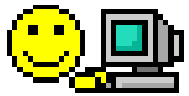
R(P2)={}　　W(P2)={b}

R(P1) ∩W(P2)={}　　R(P2) ∩W(P1)={}

W(P1) ∩W(P2)={}

R(P1)∩W(P2)∪R(P2)∩W(P1)∪W(P1)∩W(P2)={}

P1、P2可以并发执行



Bernstein条件——例2

P3：　　c=a+b

P4:　　d=c+1

R(P3)={a,b}　　W(P3)={c}

R(P4)={c}　　W(P4)={d}

R(P3) ∩W(P4)={}

R(P4) ∩W(P3)={c}

R(P3) ∩W(P4) ∪R(P4) ∩W(P3)∪W(P3)∩W(P4) ={c}

P3、P4不能并发执行

进程定义与组成

进程定义：

进程是操作系统资源分配和调度的基本单位，指在系统中正在运行的一个程序的实例，具有独立的地址空间和执行环境。

进程定义与组成

进程的组成：

进程由程序段、数据段和进程控制块（PCB）三部分组成，其中程序段存放要执行的代码，数据段存放程序运行时的变量和数据结构，PCB 则保存进程的状态和控制信息。

进程的特征：

进程具有动态性（生命周期）、并发性（多个进程同时运行）、独立性（拥有独立资源）、异步性（执行速度不可预测）和结构性（由多个部分组成）等特征。

三种基本状态

就绪状态（Ready）：

进程已获得除 CPU 外的所有必要资源，等待被调度执行。此时进程处于可运行队列中，由操作系统调度算法决定何时分配 CPU 时间片。

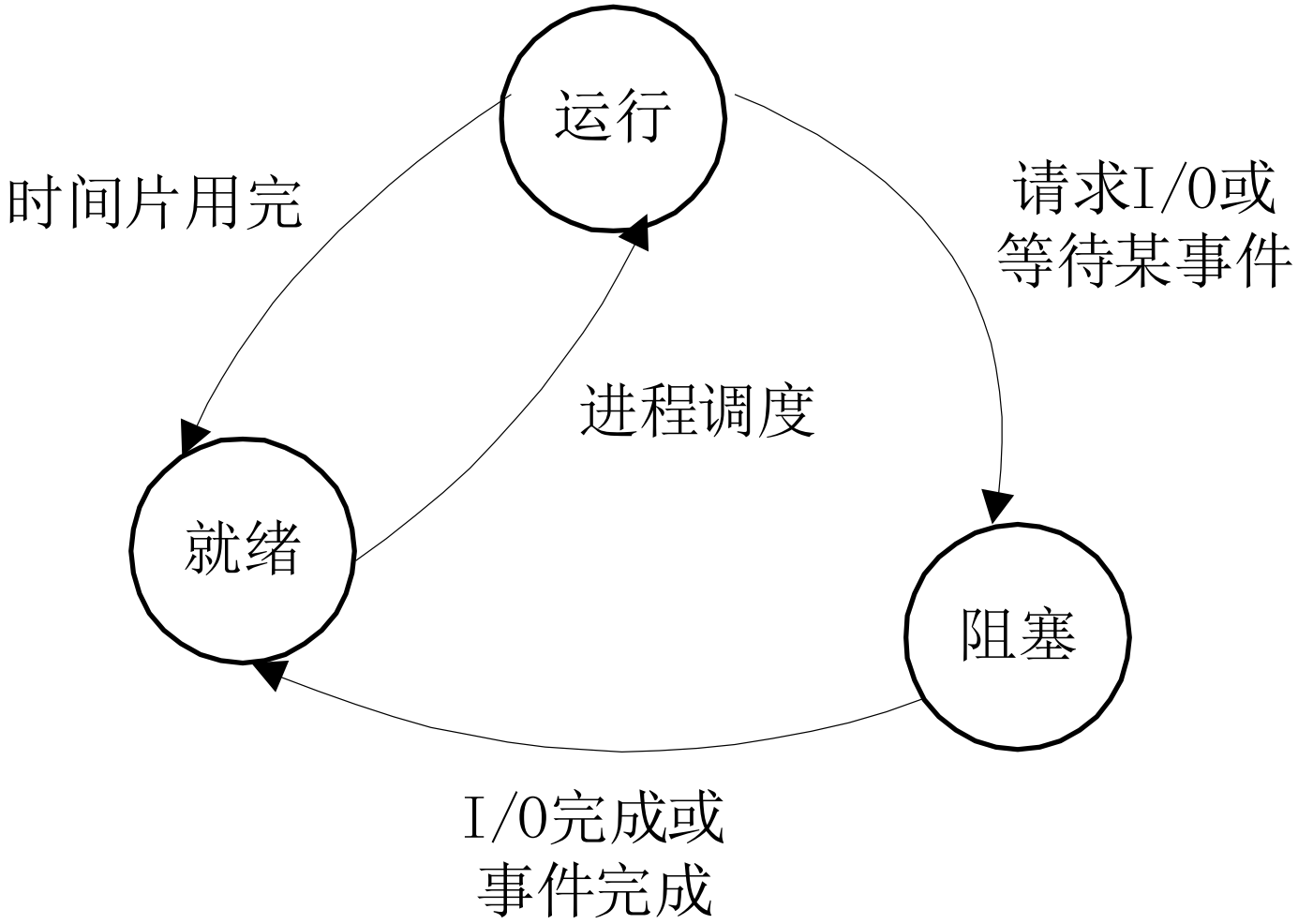
运行状态（Running）：

进程正在 CPU 上执行指令。同一时刻单核 CPU 只能有一个进程处于运行状态，多核 CPU 可并行运行多个进程。

阻塞状态（Blocked）：

进程因等待某事件（如 I/O 操作完成、信号量释放等）而暂停执行，此时即使分配 CPU 也无法继续运行，需等待事件触发后转为就绪状态。

状态转换关系



挂起状态及其特点

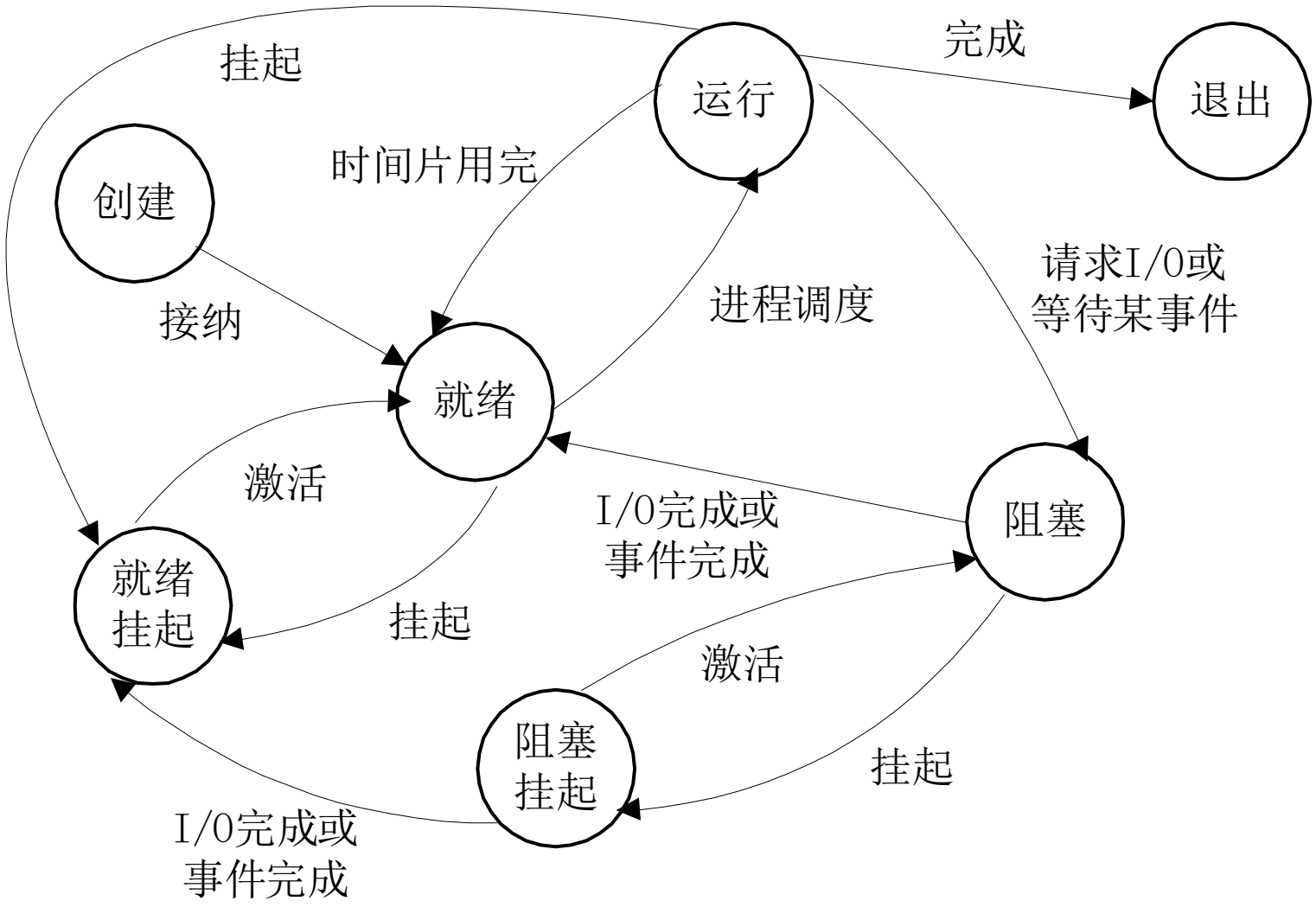
挂起与激活机制：

挂起状态是进程被临时移出内存（换出到磁盘），以释放内存资源供其他进程使用。激活时需将进程重新加载到内存，恢复执行上下文。

挂起原因：

可能因系统负载过高、用户手动挂起或调试需求触发。挂起后进程保留 PCB（进程控制块），但不再参与调度。

挂起状态及其特点

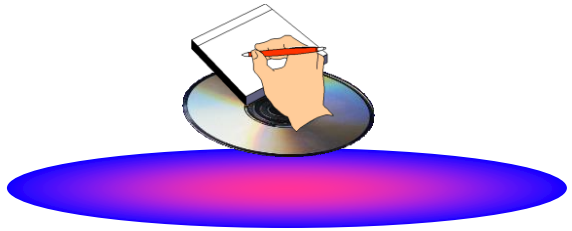

挂起
完成
运行
退出
时间片用完
创建
请求I/O或
等待某事件
进程调度
接纳
就绪
激活
阻塞
I/O完成或
事件完成
就绪
挂起
挂起
激活
阻塞
挂起
挂起
I/O完成或
事件完成

进程控制块 (PCB)

PCB 的作用：

PCB 是操作系统管理和控制进程的核心数据结构，用于记录进程的状态、标识符、程序计数器、寄存器内容、内存分配、打开文件列表等信息。

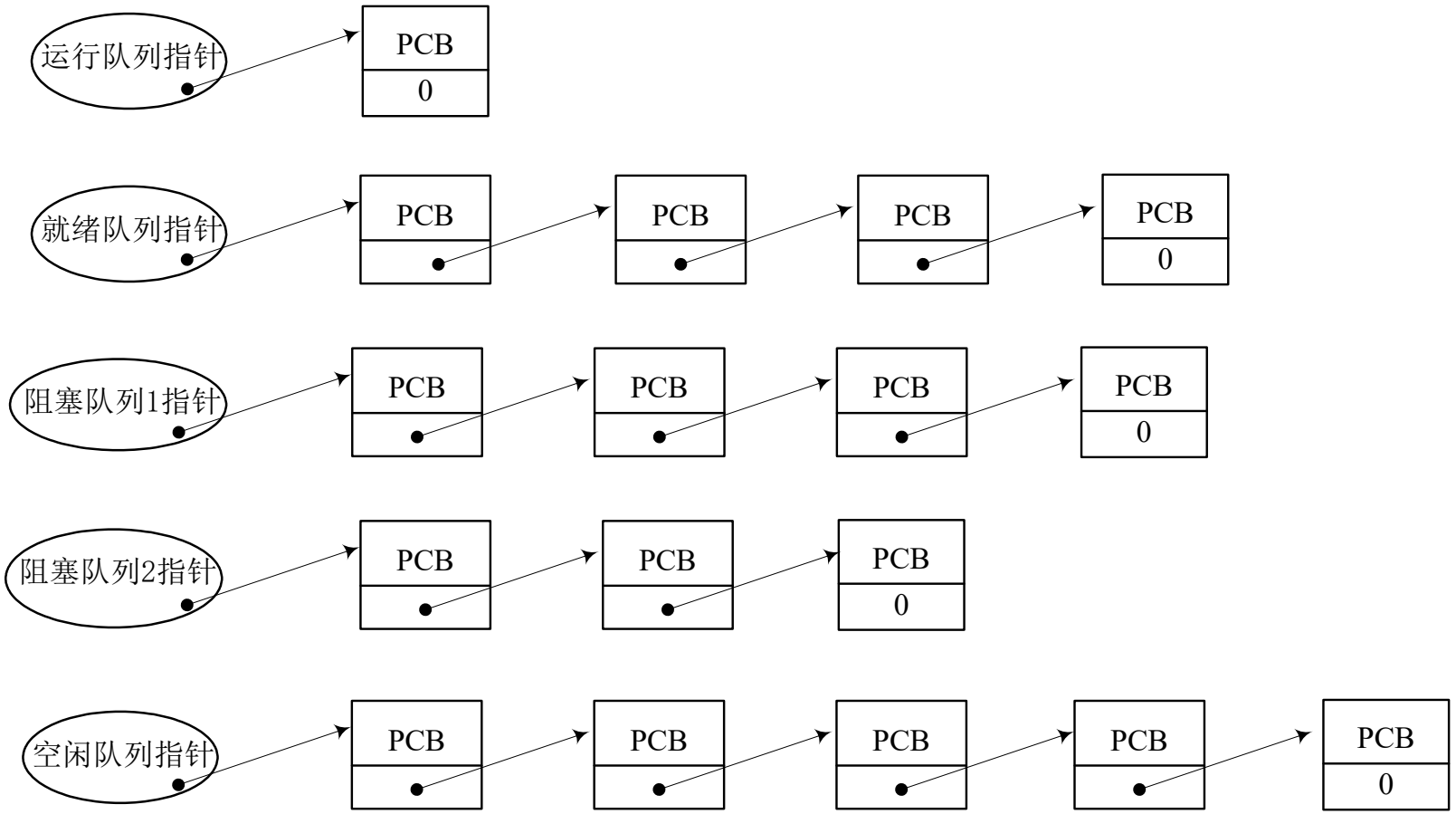
进程控制块PCB的内容



- 进程描述信息
  - 进程名
  - 进程标识符
  - 用户名
- 处理机状态信息
  - 通用寄存器
  - 指令计数器
  - 程序状态字寄存器
  - 栈指针
- 进程调度信息
  - 进程状态
  - 进程优先级
  - 运行统计信息。
  - 进程阻塞原因。
- 进程控制和资源占有量信息
  - 程序入口地址
  - 程序的外存地址
  - 进程同步及通信机制
  - 资源占有信息
  - 链接指针

进程控制块PCB的组织

●链接方式



进程控制块 (PCB)

PCB 的维护：

操作系统在进程创建时分配 PCB，在进程终止时回收 PCB，并在进程状态转换时更新 PCB 中的相应字段，确保进程管理的正确性和高效性。

操作系统内核

核心功能模块：

操作系统内核是系统中最核心的部分，负责管理硬件资源、进程调度、内存管理等关键任务，确保系统高效稳定运行。

特权执行模式：

内核运行在特权模式下，拥有直接访问硬件和所有系统资源的权限，而用户程序则运行在非特权模式下，通过系统调用与内核交互。

操作系统内核

中断处理机制：

内核负责处理硬件中断和软件异常，确保系统能够及时响应外部事件，如设备 I/O 完成或时钟中断触发的进程调度。

进程管理基础：

内核提供进程创建、销毁、切换等基本功能，维护进程控制块（PCB）和进程状态，是进程控制操作的实现基础。

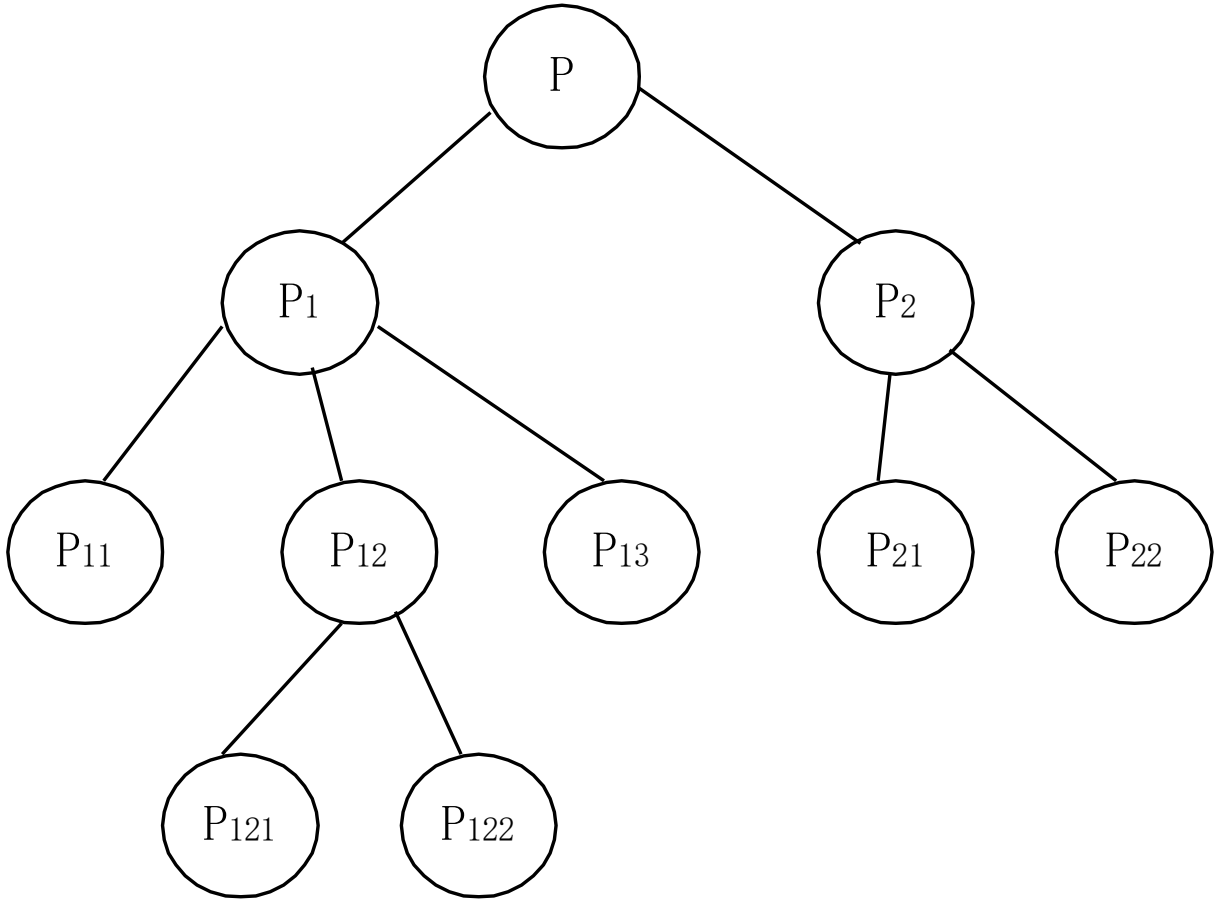
内核与原语

原语

- 原语由若干条指令构成、用于完成一定功能的过程。
- 原语是“原子操作”。即一个操作中的所有动作，要么全做，要么全不做。换言之，原子操作是一个不可分割的操作。

进程的创建与撤消

进程家族树



进程创建

引起进程创建的事件

- 用户登录
- 新作业进入系统
- 提供服务
- 应用请求

创建原语要做的工作

- 申请空白PCB
- 为进程分配资源
- 初始化PCB
  - 初始化进程描述信息
  - 初始化处理机状态信息
  - 初始化进程控制信息
- 将新进程插入就绪队列

进程的撤消

引起进程撤消的事件

- 进程正常结束
- 进程异常结束
- 外界干预

撤消原语要做的工作

- 查找撤消进程的PCB
- 若进程处于执行状态，终止之，并进行进程调度
- 若有子孙，予以终止
- 归还资源
- 从所在队列移出

进程的阻塞与唤醒

引起进程阻塞的事件

- 请求系统服务
- 启动某种操作
- 数据尚未到达
- 无新工作可做

阻塞原语要做的工作

- 停止进程的执行
- 将进程插入阻塞队列，改变进程在PCB中的状态
- 重新调度

唤醒原语要做的工作

- 将进程从阻塞队列解下
- 将进程插入就绪队列
- 改变进程在PCB中的状态

挂起与激活

1. 挂起操作场景：

   当系统内存不足或用户显式请求（如调试）时，内核将进程从内存换出到磁盘交换区，状态标记为挂起，释放其物理内存资源。

2. 挂起态细分：

   可分为阻塞挂起（进程原本阻塞且被换出）和就绪挂起（进程就绪但因资源限制被换出），影响后续激活策略。

线程的定义

轻量级执行单元：

线程是操作系统能够进行运算调度的最小单位，它被包含在进程之中，是进程中的实际运作单位。一个进程中可以并发多个线程，每条线程并行执行不同的任务。

共享进程资源：

线程与同属一个进程的其他线程共享进程所拥有的全部资源，包括代码段、数据段、堆等，但每个线程拥有独立的栈空间和程序计数器，确保执行流的独立性。

提高程序效率：

线程的引入使得程序能够更高效地利用多核处理器资源，通过并行执行任务

线程与进程的比较

01. 资源开销差异：

线程的创建、切换和销毁所需资源远少于进程，因为线程共享进程的地址空间和系统资源，而进程需要独立的资源分配，导致上下文切换成本较高。

02. 独立性对比：

进程之间相互隔离，一个进程崩溃通常不会影响其他进程；而线程共享内存空间，一个线程的错误可能导致整个进程崩溃，稳定性较低。

03. 并发粒度不同：

进程是资源分配的基本单位，线程是 CPU 调度的基本单位。多线程程序可以在单个进程内实现细粒度并发，而多进程则依赖于操作系统的进程间通信（IPC）机制。

线程实现方式

用户级线程：

由用户空间的线程库（如 POSIX 线程库）实现，内核无感知。优点是切换速度快且不依赖操作系统，缺点是单个线程阻塞会导致整个进程阻塞，无法利用多核优势。

内核级线程：

由操作系统内核直接管理，每个线程对应一个内核调度实体。优点是可真正并行执行且阻塞不影响其他线程，缺点是切换需陷入内核，开销较大。

混合模型：

结合用户级与内核级线程的优势，用户态可创建大量轻量级线程，内核态负责将这些线程映射到少量内核线程上执行。典型代表如 Go 语言的 GMP 调

多线程优势

提升资源利用率：

多线程允许单个进程内并发执行多个任务，通过共享内存空间减少上下文切换开销，显著提高 CPU 和 I/O 设备的利用率。

增强响应能力：

在交互式应用中（如 GUI 界面），多线程可将耗时任务分配到后台线程执行，避免主线程阻塞，确保用户操作的实时响应。

简化复杂任务设计：

多线程模型更贴合现实世界中的并行需求（如服务器处理多客户端请求），通过任务分解降低编程复杂度。

多线程挑战

线程同步问题：

多个线程访问共享资源时可能引发数据不一致，需依赖互斥锁、信号量等机制确保操作的原子性。

调试难度增加：

线程间执行顺序的不确定性可能导致难以复现的竞态条件，传统调试工具难以追踪多线程程序的动态行为。

性能开销：

线程创建、销毁及上下文切换会消耗系统资源，过度使用线程可能导致调度开销反而降低整体性能。

典型应用场景

高并发服务器：

Web 服务器（如 Nginx）通过多线程/多进程模型处理海量并发请求，每个线程独立响应客户端，避免单线程阻塞导致的性能瓶颈。

数据库系统（如 MySQL）利用多线程实现查询并行执行、事务隔离和日志异步写入，提升吞吐量和响应速度。

实时数据处理：

金融交易系统中，多线程用于同时处理行情接收、风险计算和订单执行，确保毫秒级延迟要求。

视频流处理框架（如 FFmpeg）通过线程池分解编码、解码、滤波任务，充分利用多核 CPU 实现实时转码。

感谢观看

THANKS